Assignment Report: Implementing Divide and Conquer Algorithms

Introduction

This report covers the implementation and analysis of two algorithms leveraging the divide-and-conquer paradigm:

1. **Strassen's Matrix Multiplication**: A method for multiplying two n*n matrices more efficiently than the conventional approach.
2. **Closest Pair of Points**: An algorithm to find the closest pair of points in a 2D plane based on their Euclidean distance.

Divide-and-conquer is a powerful strategy for solving problems by breaking them into smaller subproblems, solving each recursively, and combining their results. The divide and conquer technique significantly reduces both algorithm's time complexity from naive brute force approach. The divide and conquer implementation of both algorithms demonstrate how this technique reduces computational complexity and enhances performance.

Implementation

Part 1: Strassen's Matrix Multiplication

- **Overview**: Strassen's algorithm reduces the number of multiplications required for matrix multiplication by decomposing the problem into seven subproblems of half the size, compared to eight in the naive approach.
- **Key Steps**:

The program begins by iterating over different matrix sizes, starting from 2^1 to 2^9. For each size, two random square matrices a and b are generated using the genmat() function. These matrices are populated with random integers ranging from 0 to 999.

For each pair of matrices, the program first performs normal matrix multiplication using the mul() function, which implements the standard O(n^3) approach with three nested

loops. The time taken for this operation is recorded using the chrono library's high-resolution clock.

Next, the program performs Strassen's matrix multiplication using the strassen() function. Strassen's method recursively splits the matrices into smaller submatrices, performs seven specific matrix multiplications, and combines the results to compute the final product. This process reduces the theoretical time complexity to O(n^log7). The time taken for Strassen's multiplication is also measured.

Finally, for each matrix size, the program prints the dimensions of the matrices and the execution times for both the normal and Strassen multiplication methods. This process is repeated for all sizes, enabling a direct comparison of their performance.

- **Challenges**:
    - Handling matrices of size not equal to powers of 2 required padding with zeros. Though in my code it is never used, but is implemented for if the case occurs, it will be able to handle that.
    - Debugging recursive calls and ensuring matrix indexing consistency.

Part 2: Closest Pair of Points

- **Overview**: The problem involves finding the pair of points with the smallest Euclidean distance in a given set. The divide-and-conquer approach splits the points into subsets and combines results to achieve O(nlogn) complexity.
- **Key Steps**:

The program starts by iterating over a series of increasing point counts, ranging from one thousand to fifteen thousand, with increments of one thousand. For each iteration, it generates a set of random, unique points using the `genpoint()` function. Each point is defined by its x and y coordinates, and the points are stored in a vector.

For every set of points, the program first applies the brute force method to determine the closest pair of points. This method calculates the distance between all possible pairs of points using the distance() function and keeps track of the minimum distance along with the corresponding point pair. The time taken for this operation is recorded using the high-resolution clock from the chrono library.

Next, the program uses the divide and conquer method to find the closest pair of points. This approach sorts the points by x and y coordinates and recursively divides the points into smaller subsets, solving the problem for each subset. It also checks a strip of points near the division line to ensure the closest pair is not missed. The function combines the results from the subsets and the strip to find the overall closest pair of points. The time taken for this operation is also measured.

Finally, the program outputs the number of points, the closest pair of points, and the minimum distance for both the brute force and divide and conquer methods. It also displays the execution times for both approaches, allowing a comparison of their performance. This process is repeated for all point counts, and the results are printed to the console.

- **Challenges**:
  - Efficiently sorting and merging points based on their x and y coordinates.
  - Correctly managing edge cases like duplicate and collinear points.
  - Efficiently managing stack overflow cases since a lot of function call is happening at each step.

Performance Analysis

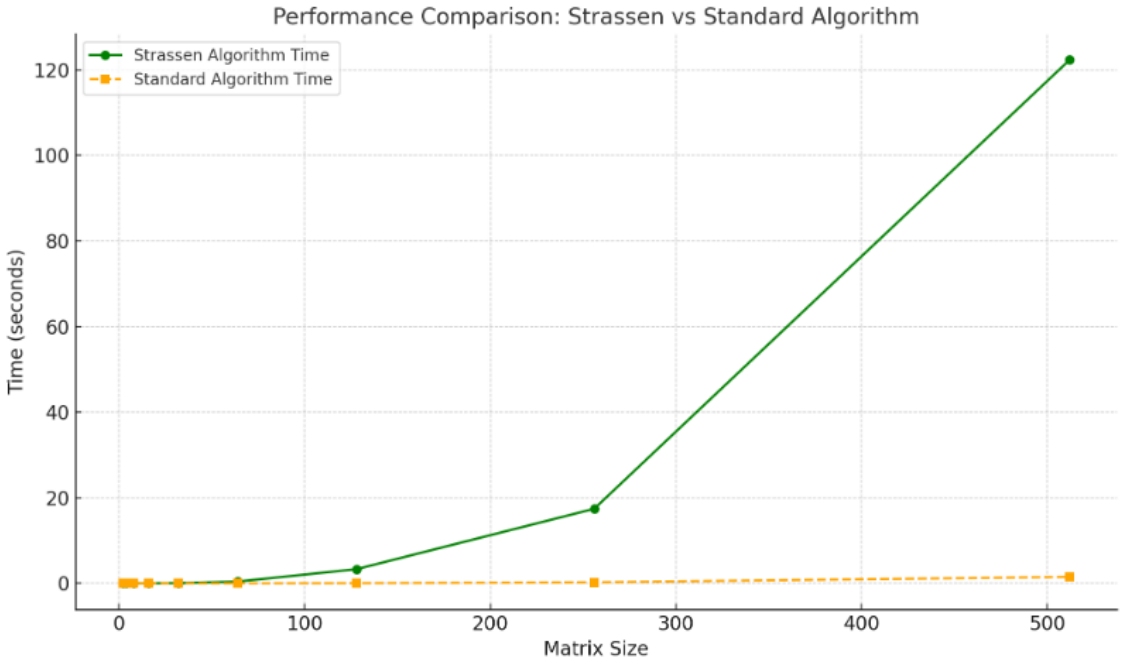
Strassen's Matrix Multiplication vs. Standard Multiplication

- **Observations**:
  Theoretically, the time complexity of strassen's multiplication is O(n^2.81),which is better compared to the standard matrix multiplication which has a complexity of O(n^3).But in reality a different phenomenon was observed.

  The time taken for strassen's multiplication was actually larger than the standard matrix multiplication. This could've happened for several reasons. Firstly, at each recursion call there are 4 subvectors of A, 4 subvectors of B and 7 vectors(i,e matrices) for strassens formula is being generated. This generation of vector takes some amount of time. Again in the creation of 7 vectors calls the sub(), add() and stassen() function several number of times, so all these function calls created overhead, potentially slowing down the algorithm. That is why strassen's algorithm was taking more time.

- **Graph**: A comparison graph was plotted to visualize the time differences.

Matrix Size	Strassen Time	Standard time
2	0	0
4	0	0
8	0	0
16	0.0077163	0
32	0.0531599	0
64	0.419416	0
128	3.31674	0.0421844
256	17.4278	0.250368
512	122.328	1.5083

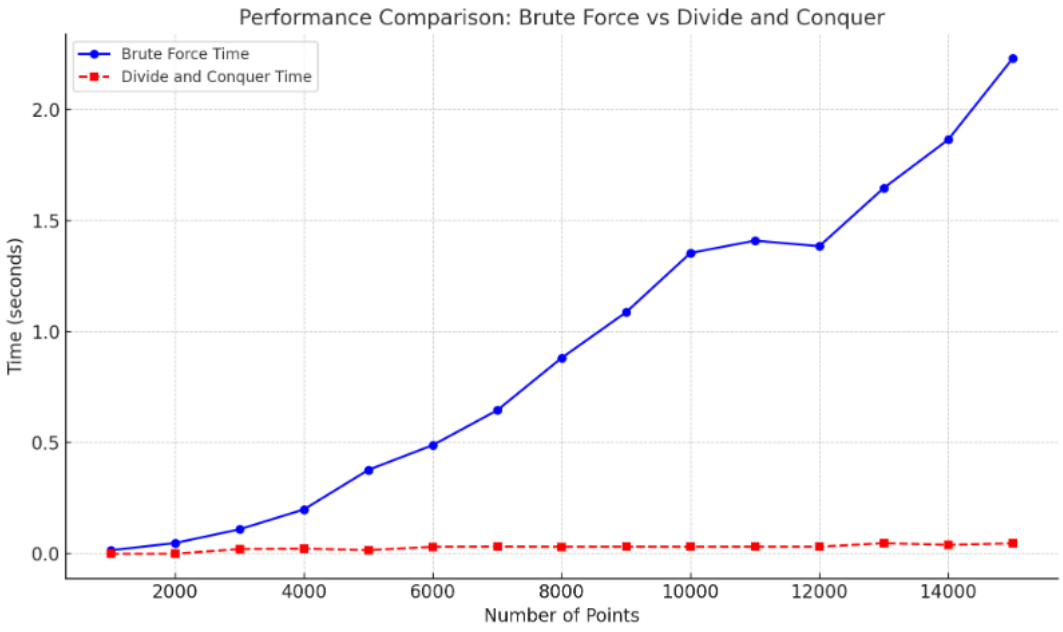


Divide and Conquer vs. Brute Force for Closest Pair

- **Observations**:
  Theoretically, the time complexity of divide and conquer approach to find closest pair of points is O(nlogn), which is better compared to the brute force approaach which has a complexity of O(n^2).
  The theoretical complexity matched with the observed time for datasets with increasing numbers of points, from 1,000 to 15,000. The divide-and-conquer method consistently outperformed brute force as the input size grew.

- **Graph**: A comparison graph was plotted to visualize the time differences.

Number of points	Brute force Time	D&Q time
1000	0.0155874	0
2000	0.0480581	0
3000	0.109967	0.0210155
4000	0.19956	0.0231534
5000	0.377503	0.016055
6000	0.489116	0.0312782
7000	0.64599	0.0321864
8000	0.881706	0.0314159
9000	1.08724	0.0316828
10000	1.35424	0.0314816
11000	1.40982	0.0317411
12000	1.38485	0.031511
13000	1.64681	0.0476664
14000	1.86454	0.0398838
15000	2.23	0.0468899



Conclusion

- **Insights**:
  - Strassen's algorithm effectively failed to reduce computational overhead for matrix multiplication by leveraging recursive decomposition, due to reasons like vector creation time, function overhead e.t.c.

- The divide-and-conquer approach for the closest pair problem improved performance and reduced the time from brute force approach but may potentially lead to stack overflow if too many function call occurs.

- **Potential Optimizations**:
  - For Strassen's algorithm, a logic could've been developed where for small matrices it will directly multiply by standard multiplication.Also, some method's could've been implemented to reduce the function overhead time.

  - The closest pair algorithm could benefit from parallel processing during the recursive divide-and-conquer steps.